TREES

Unit 7

Presented By: Saroj Bhandari

Syllabus

- 8.1 Concept and Definitions, Basic Operations in Binary Tree, Tree Height, Level and Depth
- 8.2 Binary Search Tree, Insertion, Deletion, Traversals, Search in BST
- 8.3 AVL tree and Balancing algorithm, Applications of Trees

Introduction to Tree Data Structure

- **Tree data structure** is a hierarchical structure that is used to represent and organize data in the form of parent child relationship.
- The following are some real world situations which are naturally a tree.
  - Folder structure in an operating system.
  - Tag structure in an HTML (root tag the as html tag) or XML document.
- The topmost node of the tree is called the **root**, and the nodes below it are called the child nodes. Each node can have multiple child nodes, and these child nodes can also have their own child nodes, forming a recursive structure.

Lets Look At Example

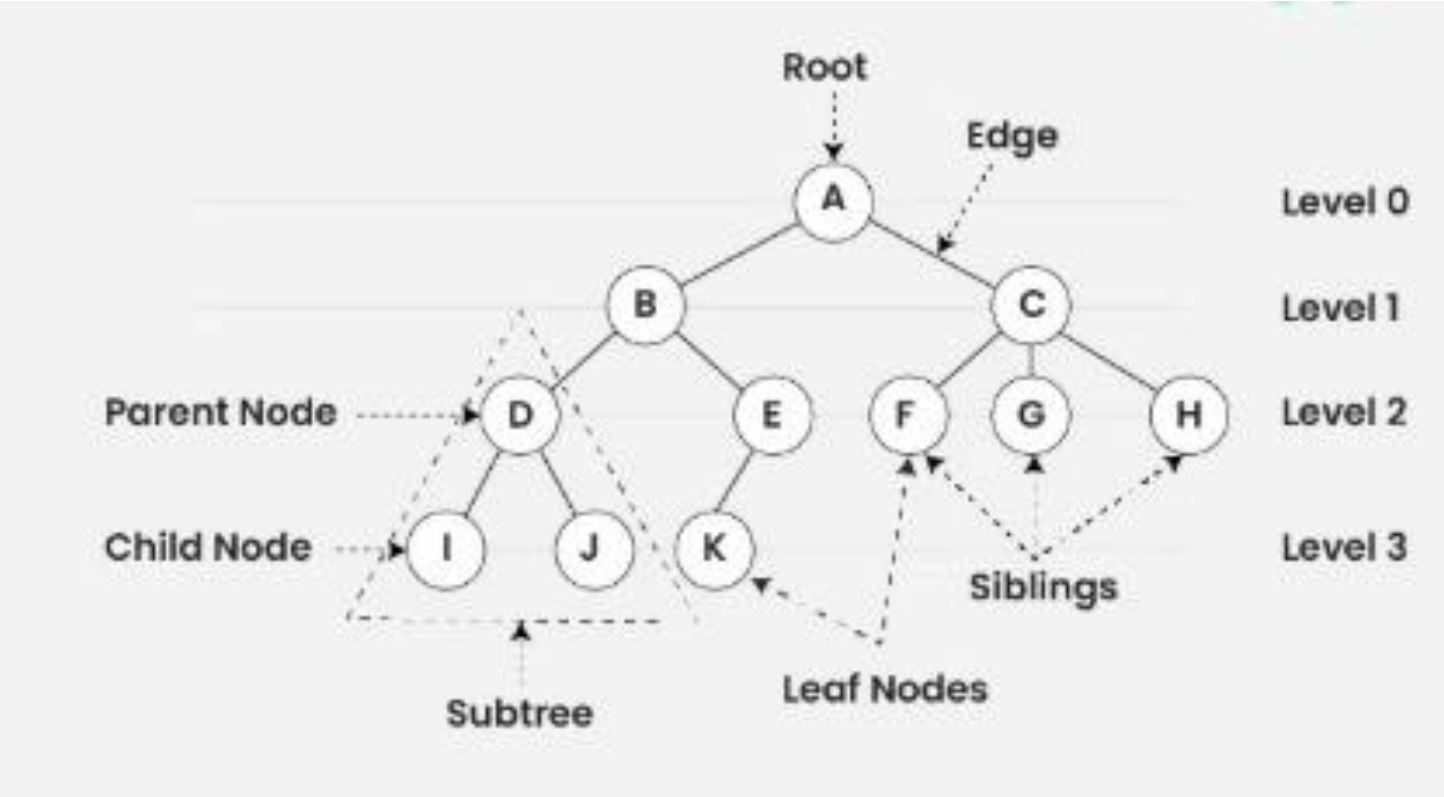


Figure: Tree Data Structure

Basic Terminologies In Tree Data Structure:

- **Parent Node:** The node which is an immediate predecessor of a node is called the parent node of that node. **{B}** is the parent node of **{D, E}**.
- **Child Node:** The node which is the immediate successor of a node is called the child node of that node. Examples: **{D, E}** are the child nodes of **{B}.**
- **Root Node:** The topmost node of a tree or the node which does not have any parent node is called the root node. {A} is the root node of the tree. A non-empty tree must contain exactly one root node and exactly one path from the root to all other nodes of the tree.
- **Leaf Node or External Node:** The nodes which do not have any child nodes are called leaf nodes. **{I, J, K, F, G, H}** are the leaf nodes of the tree.

Basic Terminologies In Tree Data Structure:

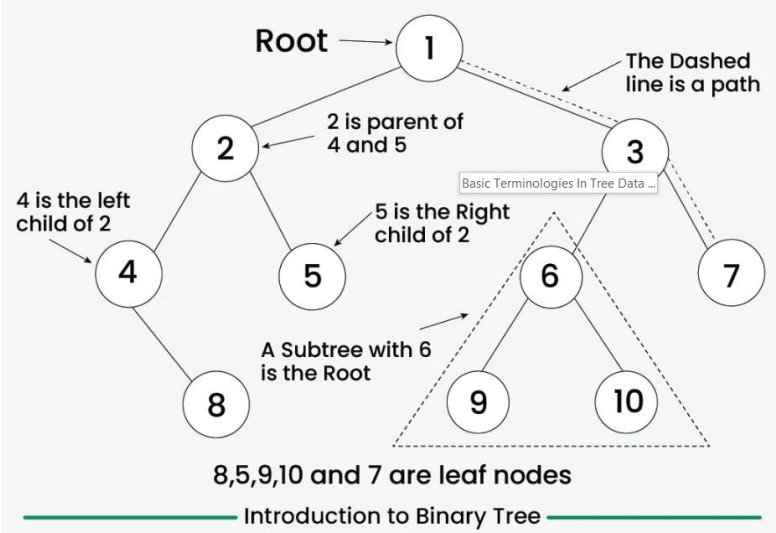
- **Ancestor of a Node:** Any predecessor nodes on the path of the root to that node are called Ancestors of that node. **{A,B}** are the ancestor nodes of the node **{E}**
- **Descendant:** A node x is a descendant of another node y if and only if y is an ancestor of x.
- **Sibling:** Children of the same parent node are called siblings. **{D,E}** are called siblings.
- **Level of a node:** The count of edges on the path from the root node to that node. The root node has level **0**.
- **Internal node:** A node with at least one child is called Internal Node.
- **Neighbour of a Node:** Parent or child nodes of that node are called neighbors of that node.
- **Subtree**: Any node of the tree along with its descendant.

Why Tree is considered a non-linear data structure?

- The data in a tree are not stored in a sequential manner i.e., they are not stored linearly. Instead, they are arranged on multiple levels or we can say it is a hierarchical structure. For this reason, the tree is considered to be a **non-linear data structure**.

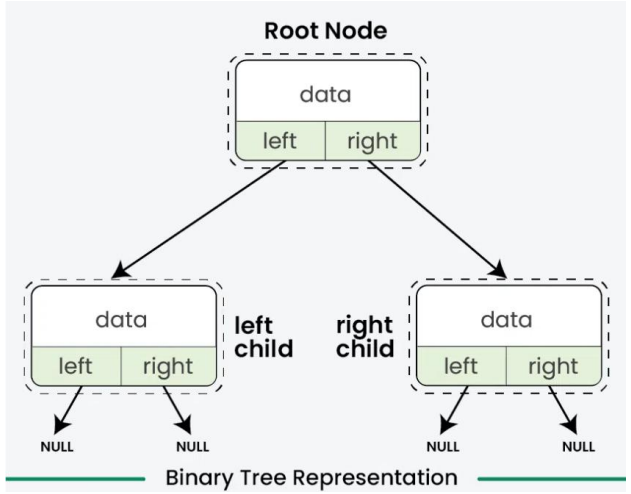
Binary Tree Data Structure

- **Binary Tree** is a **non-linear** and **hierarchical** data structure where each node has at most two children referred to as the **left child** and the **right child**.
- The topmost node in a binary tree is called the **root**, and the bottom-most nodes are called **leaves**.



Representation of Binary Tree

- Each node in a Binary Tree has three parts:
  - Data
  - Pointer to the left child
  - Pointer to the right child



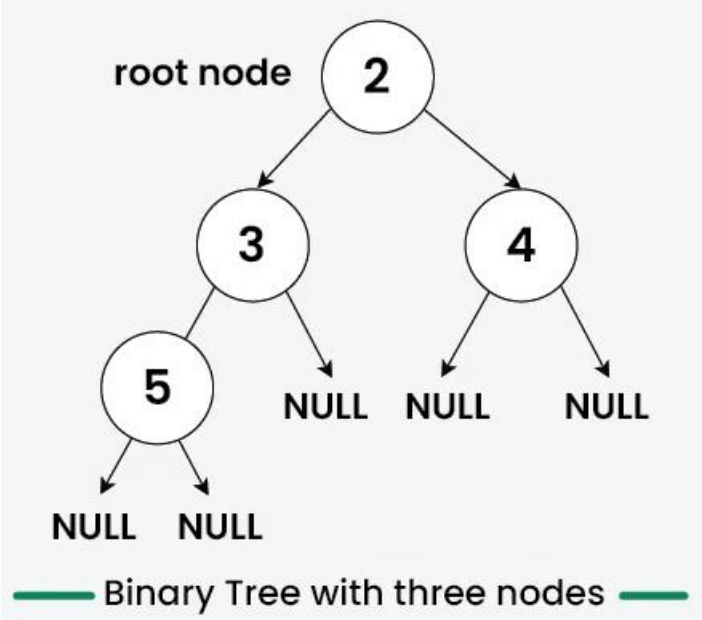
Create/Declare a Node of a Binary Tree-

```c
// Structure of each node of the tree.
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Note : Unlike other languages, C does not support
// Object Oriented Programming. So we need to write
// a separat method for create and instance of tree node
struct Node* newNode(int item) {
    struct Node* temp =
       (struct Node*)malloc(sizeof(struct Node));
    temp->key = item;
    temp->left = temp->right = NULL;
    return temp;
}
```

Example for Creating a Binary Tree

- Here's an example of creating a Binary Tree with four nodes (2, 3, 4, 5)



Example

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};

struct Node* createNode(int d) {
    struct Node* newNode =
      (struct Node*)malloc(sizeof(struct Node));
    newNode->data = d;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

int main() {
    // Initialize and allocate memory for tree nodes
    struct Node* firstNode = createNode(2);
    struct Node* secondNode = createNode(3);
    struct Node* thirdNode = createNode(4);
    struct Node* fourthNode = createNode(5);

    // Connect binary tree nodes
    firstNode->left = secondNode;
    firstNode->right = thirdNode;
    secondNode->left = fourthNode;

    return 0;
}
```

Strictly Binary Tree

- A **strictly binary tree**, also known as a **full binary tree or proper binary tree**, is a binary tree where every node has either exactly 0 or 2 children.
- This means no node can have only one child.
- **Properties:**
  - The number of leaf nodes is equal to the number of internal nodes plus one.
  - A strictly binary tree with n leaves has 2n-1 nodes.

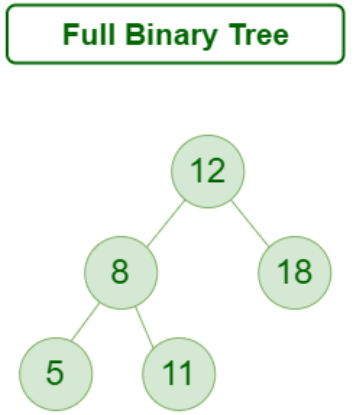
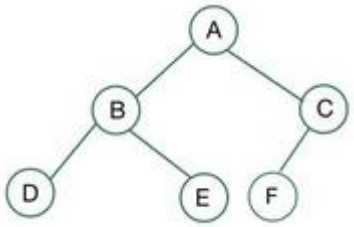


Figure: Full Binary Tree

Complete Binary Tree

- A complete binary tree is a special type of binary tree where **all the levels of the tree are filled completely** except the lowest level nodes which are filled from as left as possible.

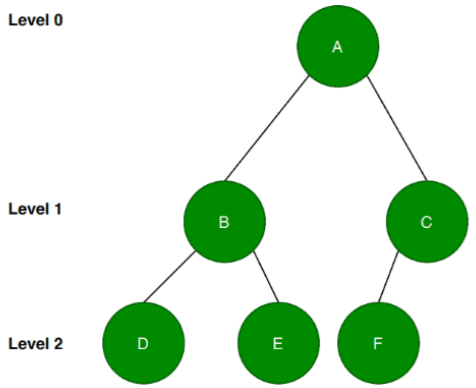
- **Properties of Complete Binary Tree:**
  - A complete binary tree is said to be a proper binary tree where all leav depth.
  - In a complete binary tree number of nodes at depth **d** is 2^d.
  - In a complete binary tree with **n** nodes height of the tree is **log(n+1)**.
  - All the levels **except the last level** are completely full.



Almost Complete Binary Tree

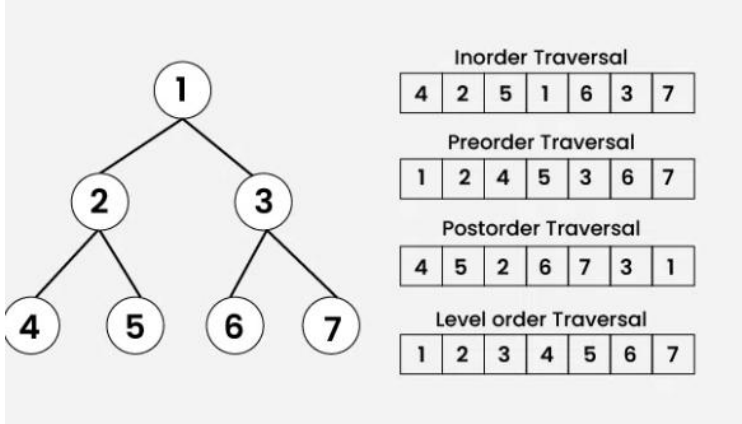
- An almost complete binary tree is a special kind of binary tree where insertion takes place level by level and from left to right order at each level and the last level is not filled fully always. It also contains nodes at each level except the last level.
- The main point here is that if we want to insert any node at the lowest level of the tree, it should be inserted from the left. All the internal nodes should be completely filled.

Example

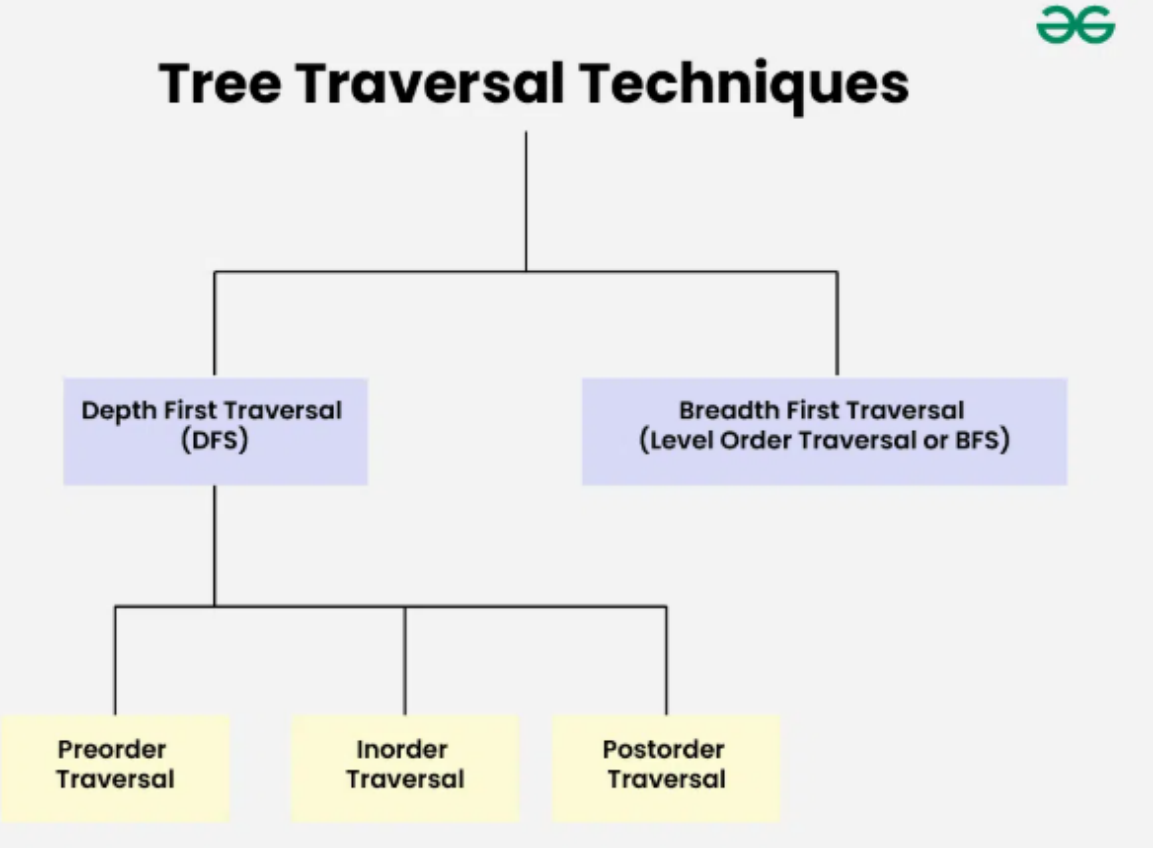


Binary Tree Traversal Techniques

- **Tree Traversal techniques** include various ways to visit all the nodes of the tree. Unlike linear data structures (Array, Linked List, Queues, Stacks, etc) which have only one logical way to traverse them, trees can be traversed in different ways.
- In this article, we will discuss all the tree traversal techniques along with their uses.



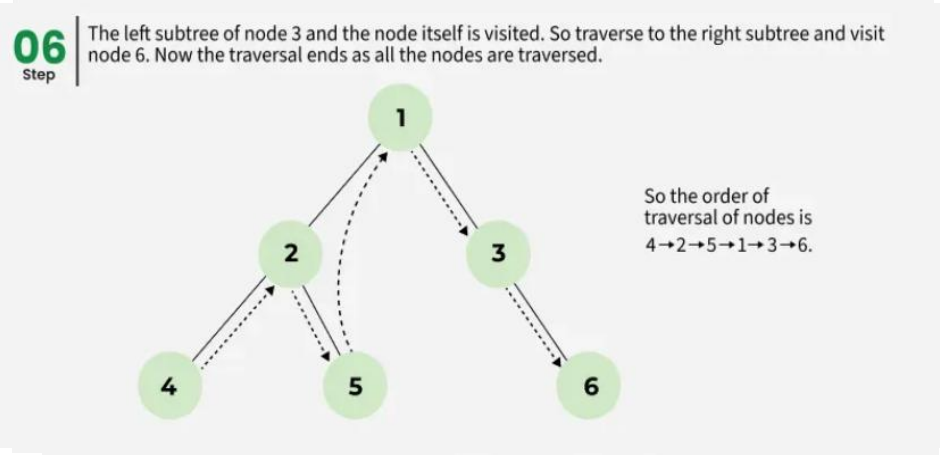
Tree Traversal



Presented By: Saroj Bhandari

Inorder Traversal

- Inorder traversal visits the node in the order: **Left -> Root -> Right**
- **Algorithm for Inorder Traversal**
  - *Traverse the left subtree, i.e., call Inorder(left->subtree)*
  - *Visit the root.*
  - *Traverse the right subtree, i.e., call Inorder(right->subtree)*

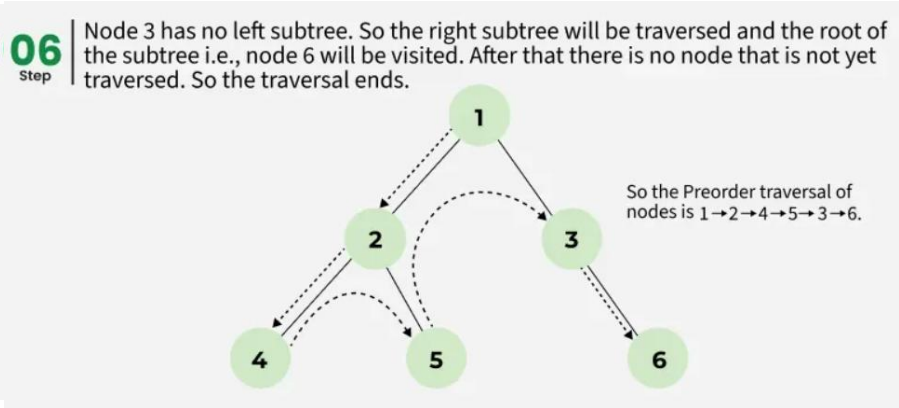


Uses of Inorder Traversal

- In the case of binary search trees (BST), Inorder traversal gives nodes in non-decreasing order.
- To get nodes of BST in non-increasing order, a variation of Inorder traversal where Inorder traversal is reversed can be used.
- Inorder traversal can be used to evaluate arithmetic expressions stored in expression trees.

Preorder Traversal

- Preorder traversal visits the node in the order: Root -> Left -> Right
- Algorithm for Preorder Traversal
- Preorder(tree)
  - Visit the root.
  - Traverse the left subtree, i.e., call Preorder(left->subtree)
  - Traverse the right subtree, i.e., call Preorder(right->subtree)

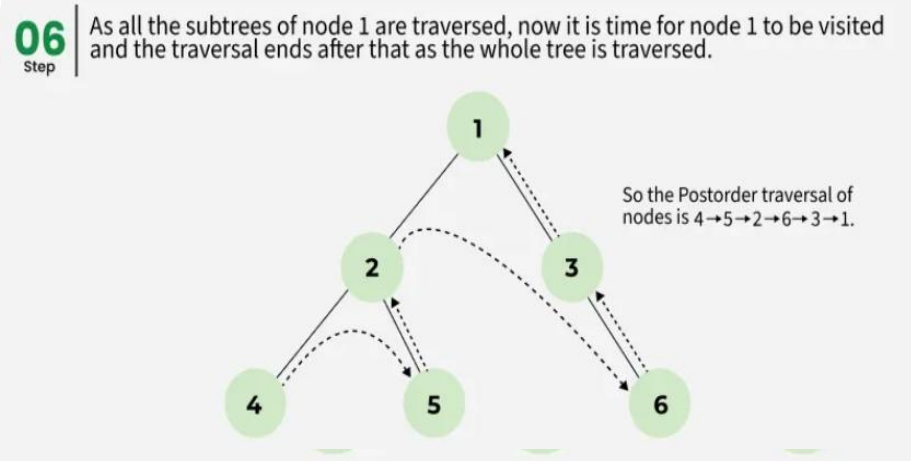


Uses of Preorder Traversal

- Preorder traversal is used to create a copy of the tree.
- Preorder traversal is also used to get prefix expressions on an expression tree.

Postorder Traversal

- Postorder traversal visits the node in the order: **Left -> Right -> Root**
- **Algorithm for Postorder Traversal:**
  - *Traverse the left subtree, i.e., call Postorder(left->subtree)*
  - *Traverse the right subtree, i.e., call Postorder(right->subtree)*
  - *Visit the root*



Uses of Postorder Traversal

- **Postorder** traversal is used to delete the tree.
- **Postorder** traversal is also useful to get the postfix expression of an expression tree.
- **Postorder** traversal can help in garbage collection algorithms, particularly in systems where manual memory management is used.

Expression Tree

- The expression tree is a binary tree in which each internal node corresponds to the operator and each leaf node corresponds to the operand.
- so for example expression tree for 3 + ((5+9)*2) would be:

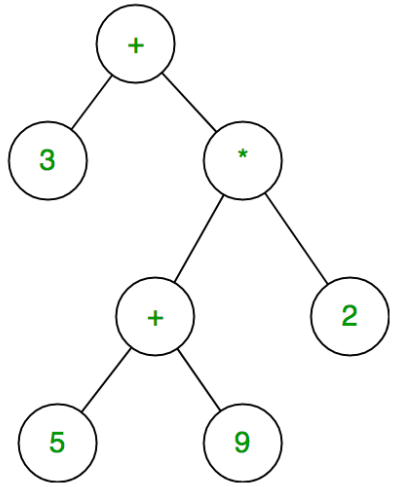
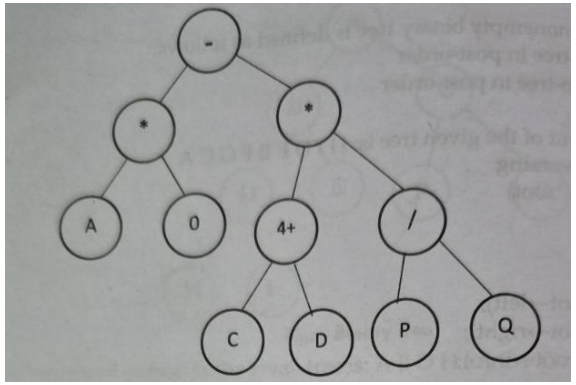


Figure Expression Tree

Now It's Your Turn

Q) Construct an expression tree of the given infix notation:

A*B-(C+D)*(P/Q)



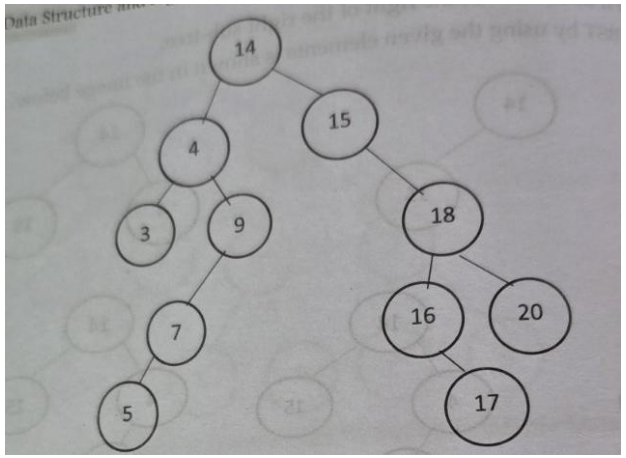
Binary Search Tree

- A **Binary Search Tree (or BST)** is a data structure used in computer science for organizing and storing data in a sorted manner.
- Each node in a **Binary Search Tree** has at most two children, a **left** child and a **right** child, with the **left** child containing values less than the parent node and the **right** child containing values greater than the parent node.
- This hierarchical structure allows for efficient **searching**, **insertion**, and **deletion** operations on the data stored in the tree.

Let's Construct a BST

Construct a BST from the following data:

14, 15, 4, 9, 7,18, 3,5,16,4,20, 17, 9



Advantage of Using BST

- Provides **efficient search, insertion, and deletion operations** (O(log n) time when balanced)
- Maintains elements in **sorted order** using in-order traversal
- **Grows and shrinks** dynamically without needing memory reallocation
- Supports efficient range queries
- Allows **quick access to minimum and maximum elements**
- Is flexible and can be extended into advanced balanced trees like **AVL or Red-Black Trees**
- Is **simpler to implement** compared to more complex data structures
- Helps **organize data hierarchically** for faster retrieval

Operations on Binary Search Key

1. **Search(k, T):**

`k` = key to search.

`T` = root of the BST.

If `T` is NULL, key `k` is not found.

If `T.key == k`, return the node (key found).

If `k < T.key`, search in the left subtree.

If `k > T.key`, search in the right subtree.

Operations on Binary Search Key

2. Insert(K,T)

`k` = key to insert.

`T` = root of the BST.

If `T` is NULL, create a new node with key `k`.

If `k < T.key`, insert into the left subtree.

If `k > T.key`, insert into the right subtree.

If `k == T.key`, do nothing or handle duplicates (based on rules).

Operations on Binary Search Key

3. Delete(k, T)

`k` = key to delete.

`T` = root of the BST.

If `T` is NULL, key `k` is not found (nothing to delete).

If `k < T.key`, delete `k` from left subtree.

If `k > T.key`, delete `k` from right subtree.

If `T.key == k`,

- If `T` has no children (leaf node), delete `T`.
- If `T` has one child, replace `T` with its child.
- If `T` has two children,
  - Find the minimum node in the right subtree (in-order successor),
  - Replace `T.key` with successor's key,
  - Delete the successor node from right subtree.

Operations on Binary Search Key

- **FindMin(T)**

  `T` = root of the BST.

  If `T` is NULL, the tree is empty.

  While `T.left` is not NULL, move to the left child.

  When `T.left == NULL`, `T` is the node with minimum key.

- **FindMax(T)**

  `T` = root of the BST.

  If `T` is NULL, the tree is empty.

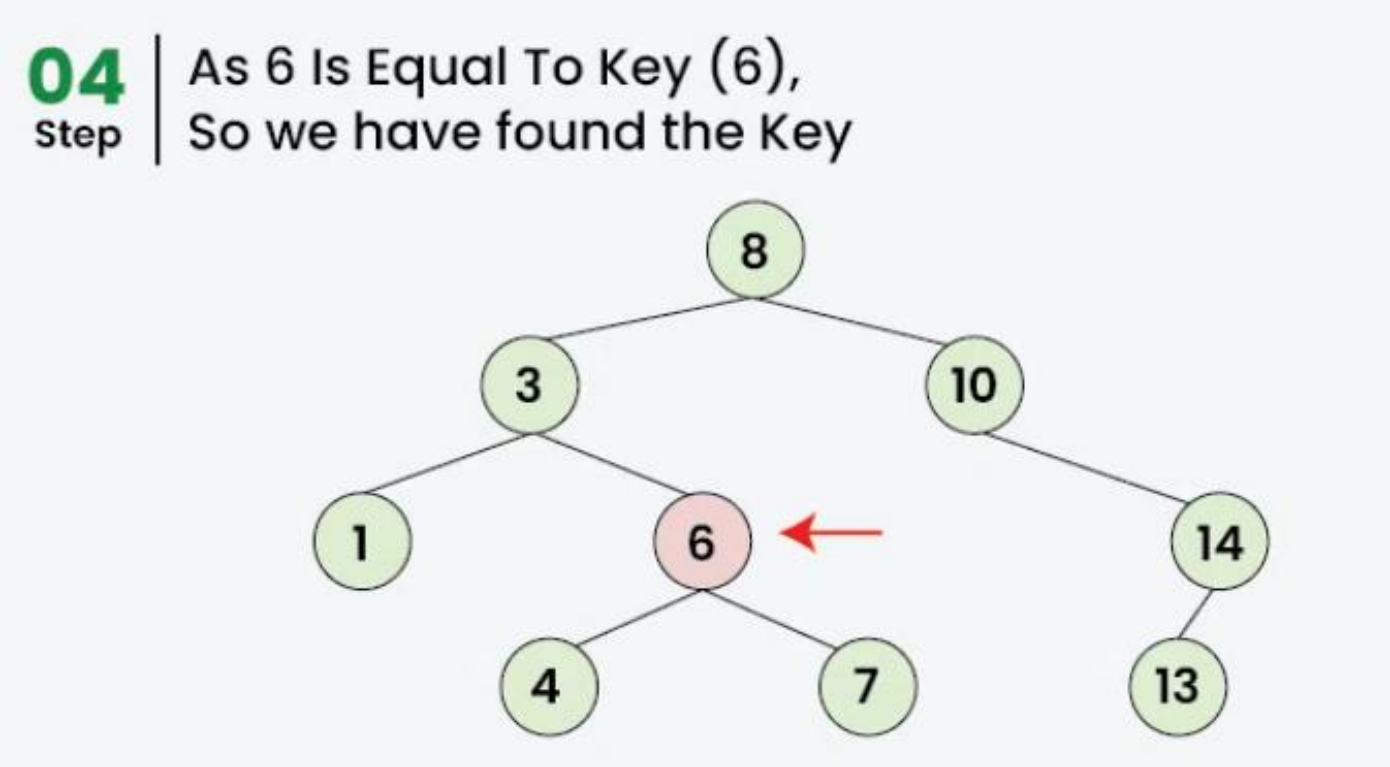
  While `T.right` is not NULL, move to the right child.

  When `T.right == NULL`, `T` is the node with maximum key.

Algorithm for BST Searching

- **Algorithm to search for a key in a given Binary Search Tree:**
- Let's say we want to search for the number **X,** We start at the root. Then:
- We compare the value to be searched with the value of the root.
  - If it's equal we are done with the search if it's smaller we know that we need to go to the left subtree because in a binary search tree all the elements in the left subtree are smaller and all the elements in the right subtree are larger.
- Repeat the above step till no more traversal is possible
- If at any iteration, key is found, return True. Else False.

Illustration of searching in a BST:



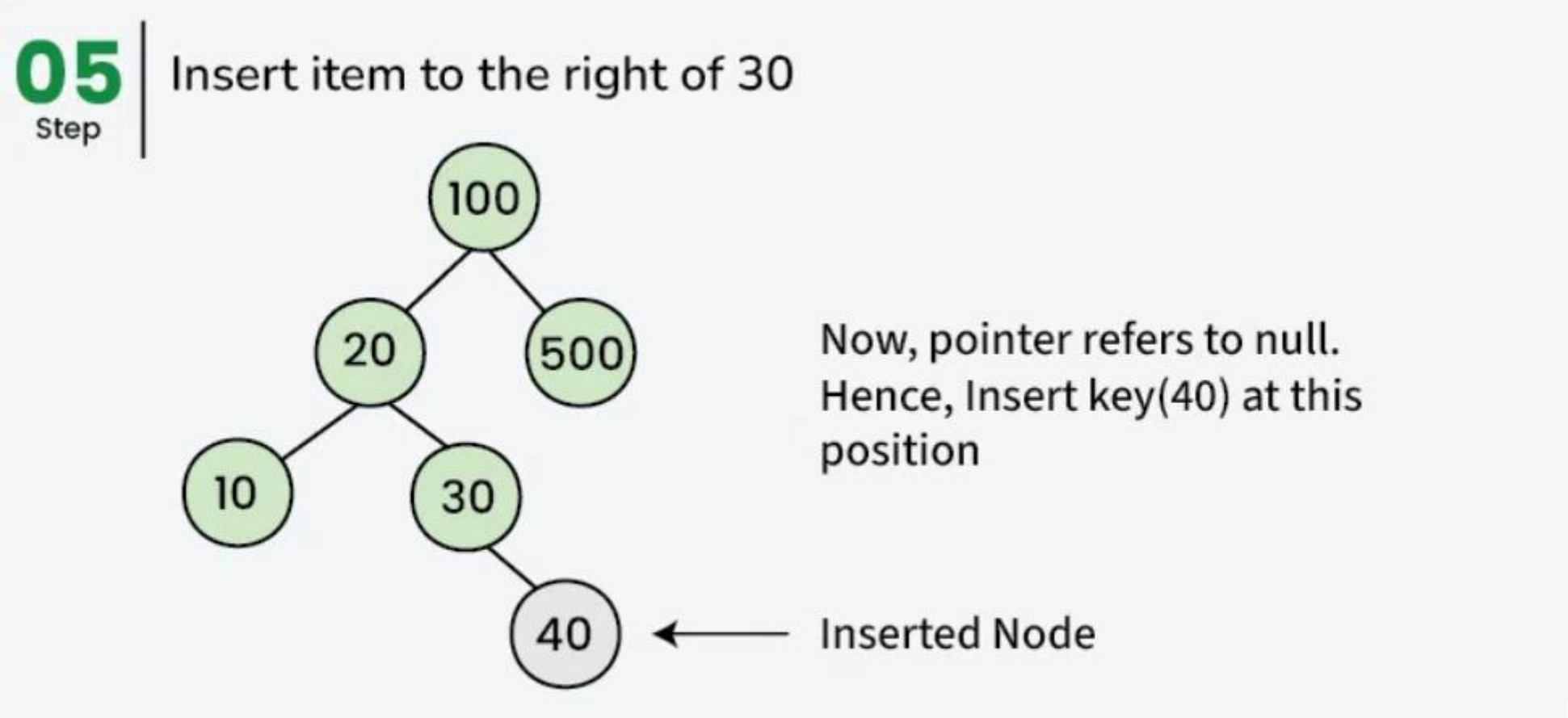
Code Implementation

- **In Lab**

Insertion of Node in BST

- The below steps are followed while we try to insert a node into a binary search tree:
  - Initilize the current node (say, **currNode or node**) with root node
  - Compare the **key** with the current node.
  - **Move left** if the **key** is less than or equal to the current node value.
  - **Move right** if the **key** is greater than current node value.
  - Repeat steps 2 and 3 until you reach a leaf node.
  - Attach the **new key** as a left or right child based on the comparison with the leaf node's value.

Illustration for Better Understanding



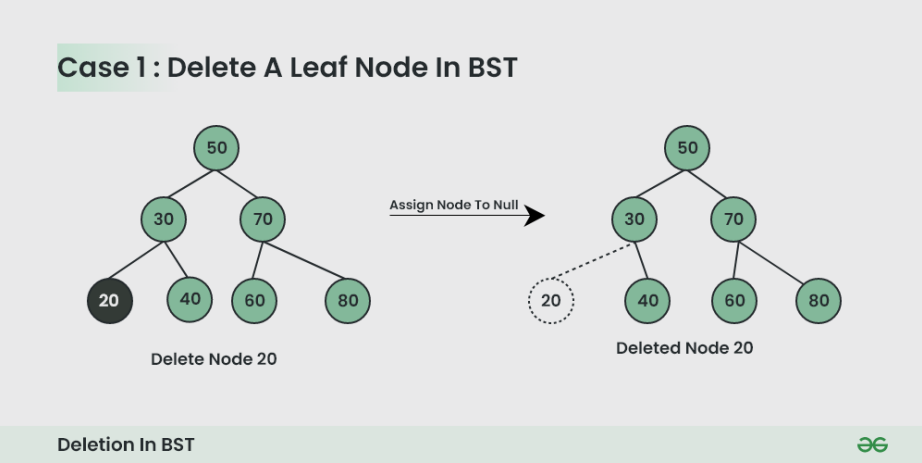
Insertion in Binary Search Tree using Recursion:

- In Lab

Deleting a Node from BST

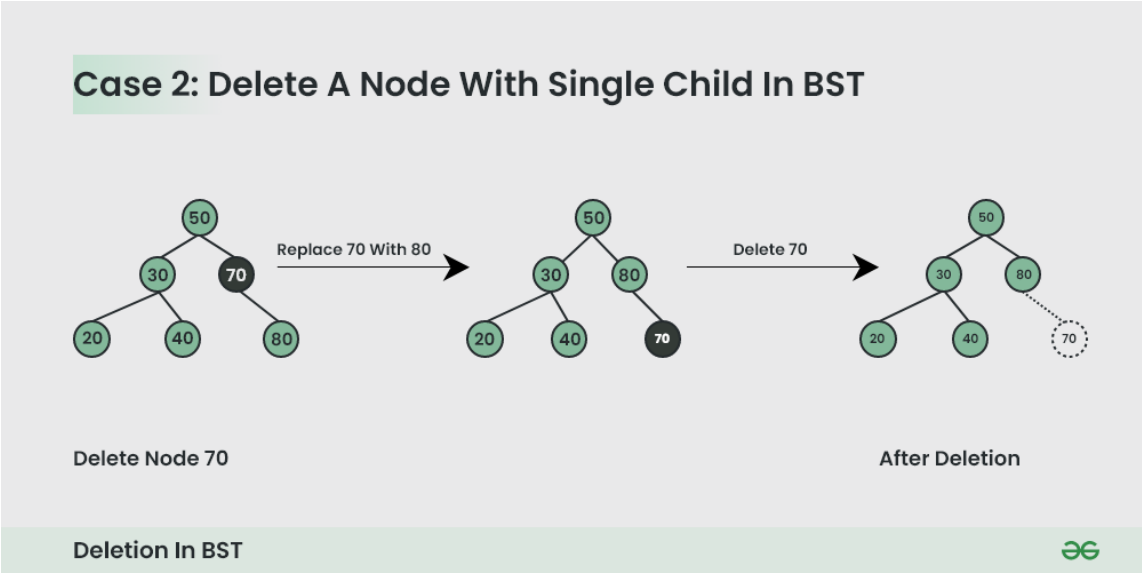
- Given a **BST**, the task is to delete a node in this **BST**, which can be broken down into 3 scenarios:

Case 1. Delete a Leaf Node in BST



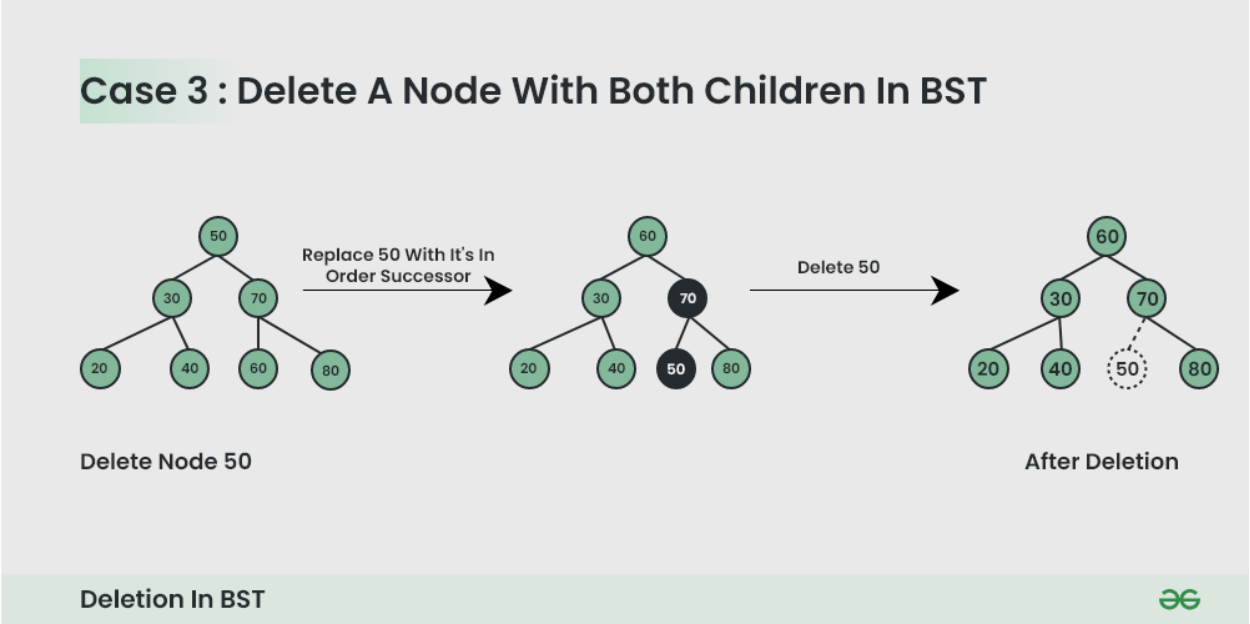
Deleting a Node from BST

Case 2. Delete a Node with Single Child in BST



Deleting a Node from BST

Case 3. Delete a Node with Both Children in BST



Code implementing for Deleting Nodes form BST

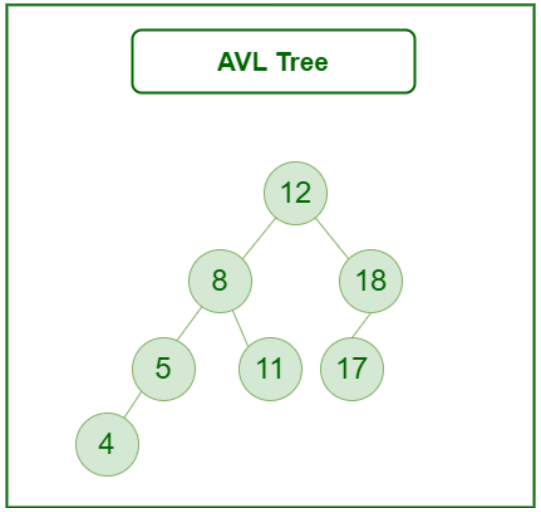
- In Lab

AVL Tree

- An **AVL tree** defined as a self-balancing **Binary Search Tree (BST)** where the difference between heights of left and right subtrees for any node cannot be more than one.
- The absolute difference between the heights of the left subtree and the right subtree for any node is known as the **balance factor** of the node. The balance factor for all nodes must be less than or equal to 1.
- Every AVL tree is also a Binary Search Tree (Left subtree values Smaller and Right subtree values greater for every node), but every BST is not AVL Tree. For example, the second diagram below is not an AVL Tree.
- The main advantage of an AVL Tree is, the time complexities of all operations (search, insert and delete, max, min, floor and ceiling) become O(Log n). This happens because height of an AVL tree is bounded by O(Log n). In case of a normal BST, the height can go up to O(n).
- An AVL tree maintains its height by doing some extra work during insert and delete operations. It mainly uses rotations to maintain both BST properties and height balance.
- There exist other self-balancing BSTs also like Red Black Tree. Red Black tree is more complex, but used more in practice as it is less restrictive in terms of left and right subtree height differences.

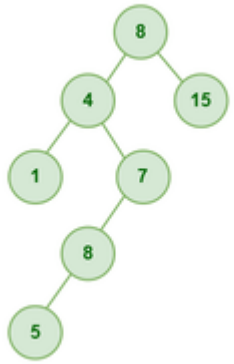
Example of an AVL Tree:

- The balance factors for different nodes are : 12 :1, 8:1, 18:1, 5:1, 11:0, 17:0 and 4:0. Since all differences are less than or equal to 1, the tree is an AVL tree.



Example of a BST which is NOT AVL:

- The Below Tree is **NOT an AVL Tree** as **t**he balance factor for nodes 8, 4 and 7 is more than 1.

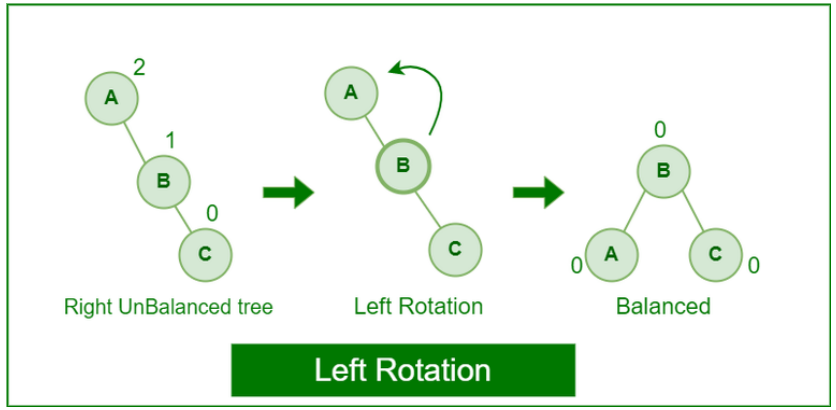


Operations on an AVL Tree:

- **Searching** : It is same as normal Binary Search Tree (BST) as an AVL Tree is always a BST. So we can use the same implementation as BST. The advantage here is time complexity is O(Log n)
- **Insertion** : It does rotations along with normal BST insertion to make sure that the balance factor of the impacted nodes is less than or equal to 1 after insertion
- **Deletion** : It also does rotations along with normal BST deletion to make sure that the balance factor of the impacted nodes is less than or equal to 1 after deletion.

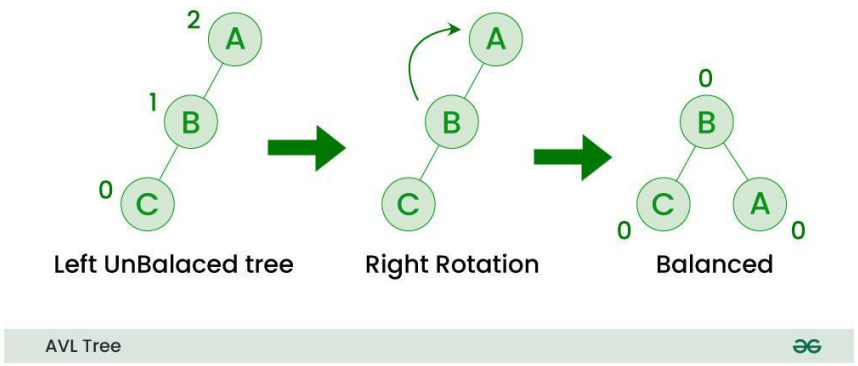
Rotating the subtrees (Used in Insertion and Deletion)

- An AVL tree may rotate in one of the following four ways to keep itself balanced while making sure that the BST properties are maintained.
- **Left Rotation**:
- When a node is added into the right subtree of the right subtree, if the tree gets out of balance, we do a single left rotation.



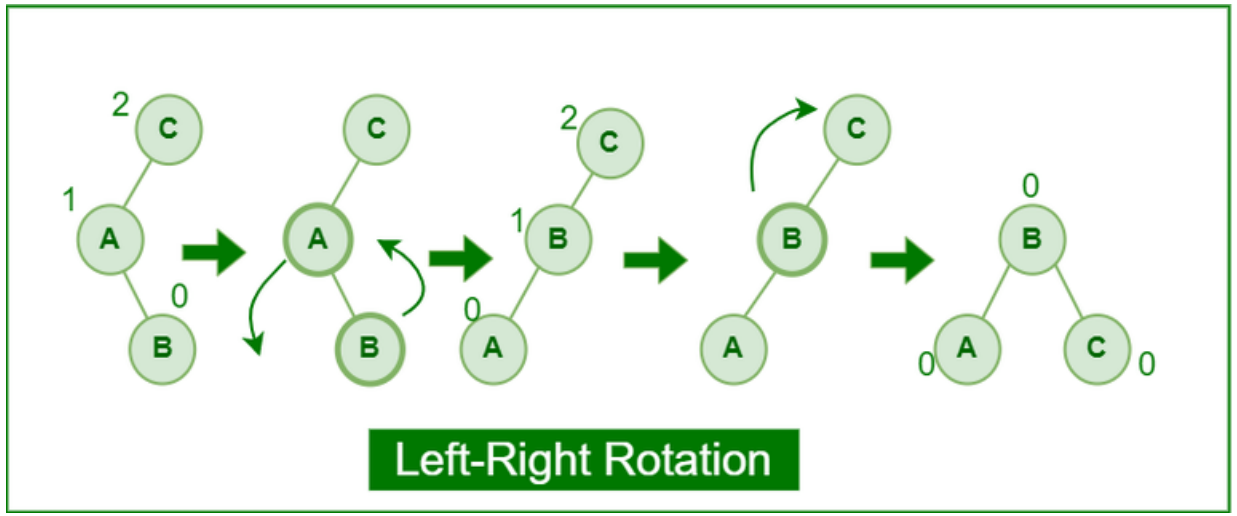
Rotating the subtrees (Used in Insertion and Deletion)

- **Right Rotation**:
- If a node is added to the left subtree of the left subtree, the AVL tree may get out of balance, we do a single right rotation.



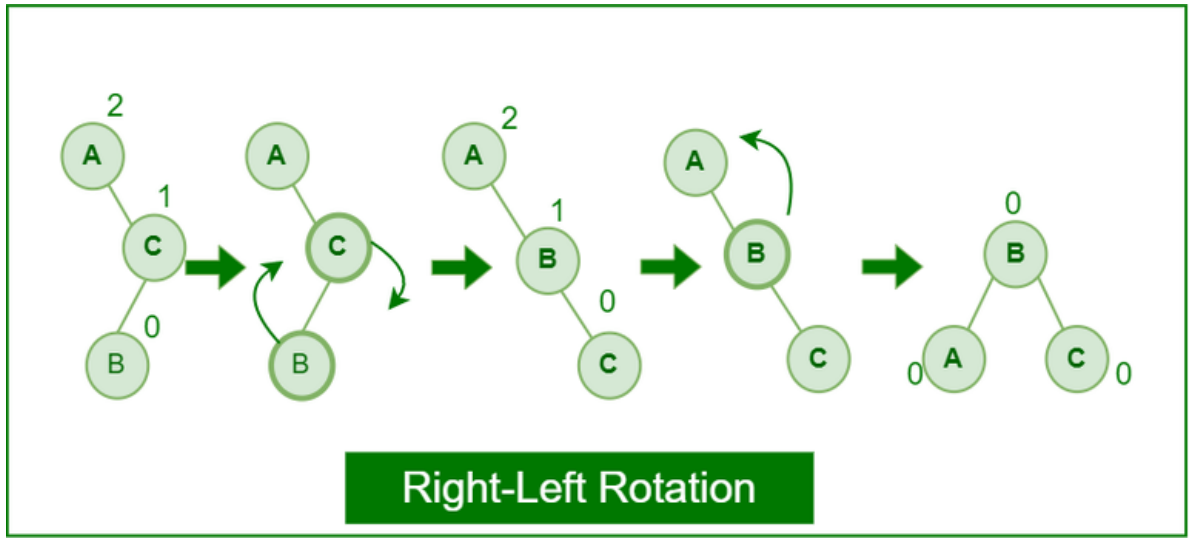
Rotating the subtrees (Used in Insertion and Deletion)

- **Left-Right Rotation**:
- A left-right rotation is a combination in which first left rotation takes place after that right rotation executes.



Rotating the subtrees (Used in Insertion and Deletion)

- **Right-Left Rotation:**
- A right-left rotation is a combination in which first right rotation takes place after that left rotation executes.



Advantages of AVL Tree:

- AVL trees can self-balance themselves and therefore provides time complexity as O(Log n) for search, insert and delete.
- It is a BST only (with balancing), so items can be traversed in sorted order.
- Since the balancing rules are strict compared to Red Black Tree, AVL trees in general have relatively less height and hence the search is faster.
- AVL tree is relatively less complex to understand and implement compared to Red Black Trees.

Disadvantages of AVL Tree:

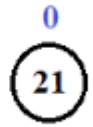
- It is difficult to implement compared to normal BST and easier compared to Red Black
- Less used compared to Red-Black trees. Due to its rather strict balance, AVL trees provide complicated insertion and removal operations as more rotations are performed.

Applications of AVL Tree:

- AVL Tree is used as a first example self balancing BST in teaching DSA as it is easier to understand and implement compared to Red Black
- Applications, where insertions and deletions are less common but frequent data lookups along with other operations of BST like sorted traversal, floor, ceil, min and max.
- Red Black tree is more commonly implemented in language libraries like **map in C++, set in C++, TreeMap in Java and TreeSet in Java.**
- AVL Trees can be used in a real time environment where predictable and consistent performance is required.

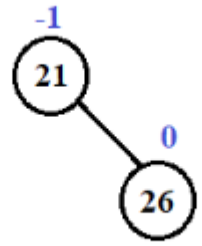
Step-by-Step Construction of the AVL Tree for the given Sequence 21, 26, 30, 9, 4, 14, 28, 18,15,10, 2, 3, 7

Step 1 - Insert 21



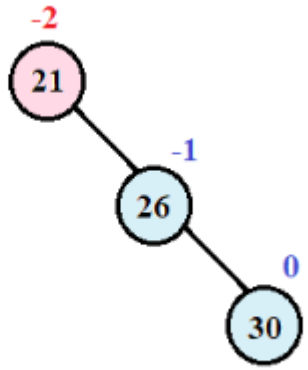
Tree is Balanced

Step 2 - Insert 26



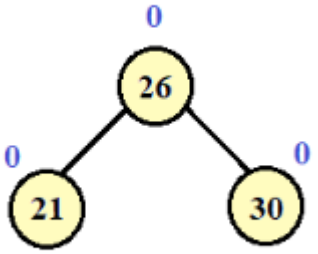
Tree is Balanced

Step 3 - Insert 30



Tree is Not Balanced, Need a Rotation

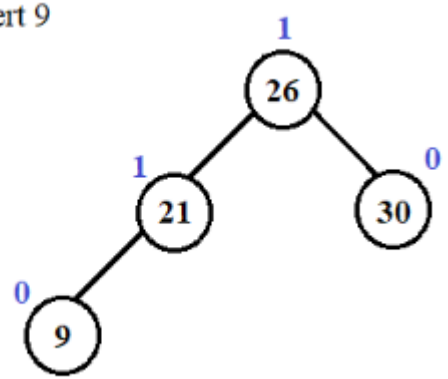
LL Rotation



Tree is Balanced

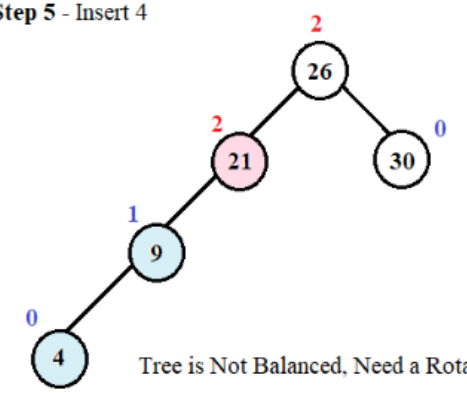
AVL Tree Example

Step 4 - Insert 9



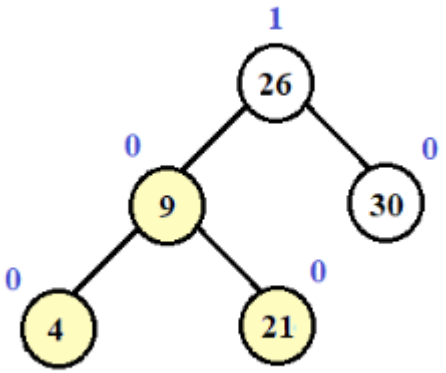
Tree is Balanced

Step 5 - Insert 4



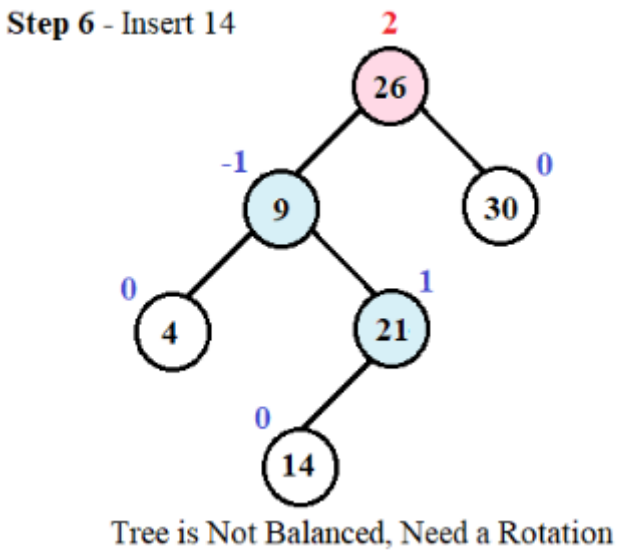
Tree is Not Balanced, Need a Rotation

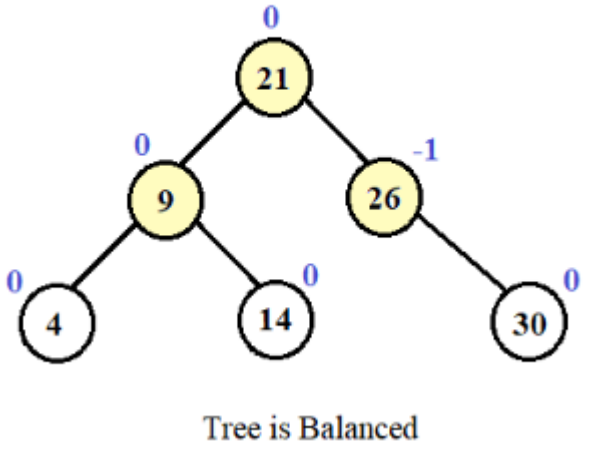
RR Rotation



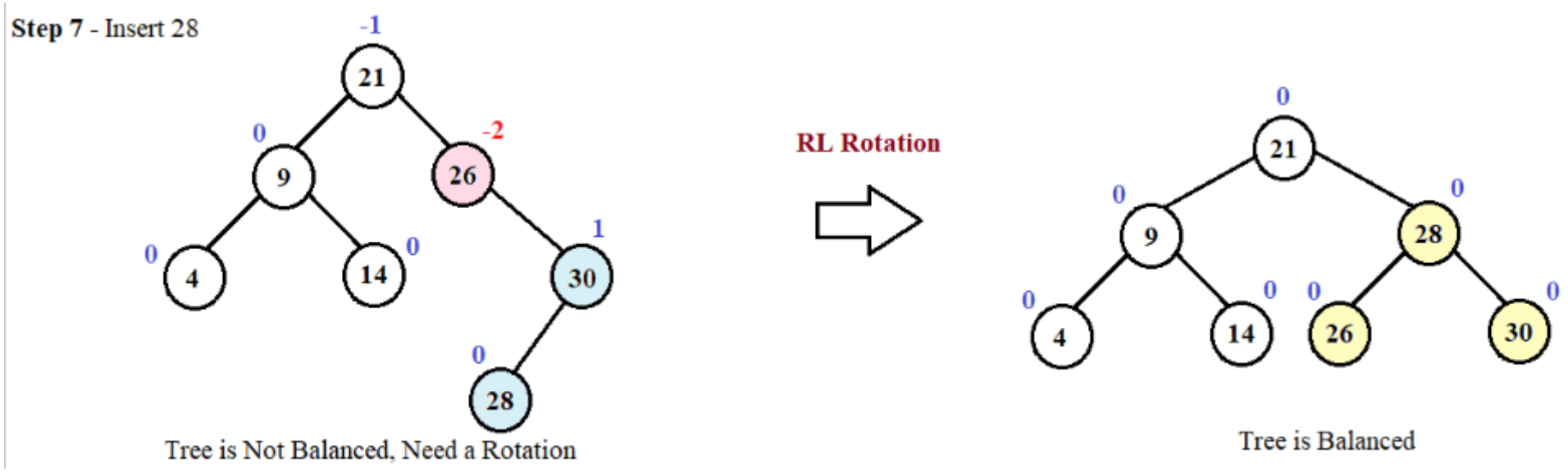
Tree is Balanced

AVL Tree Example

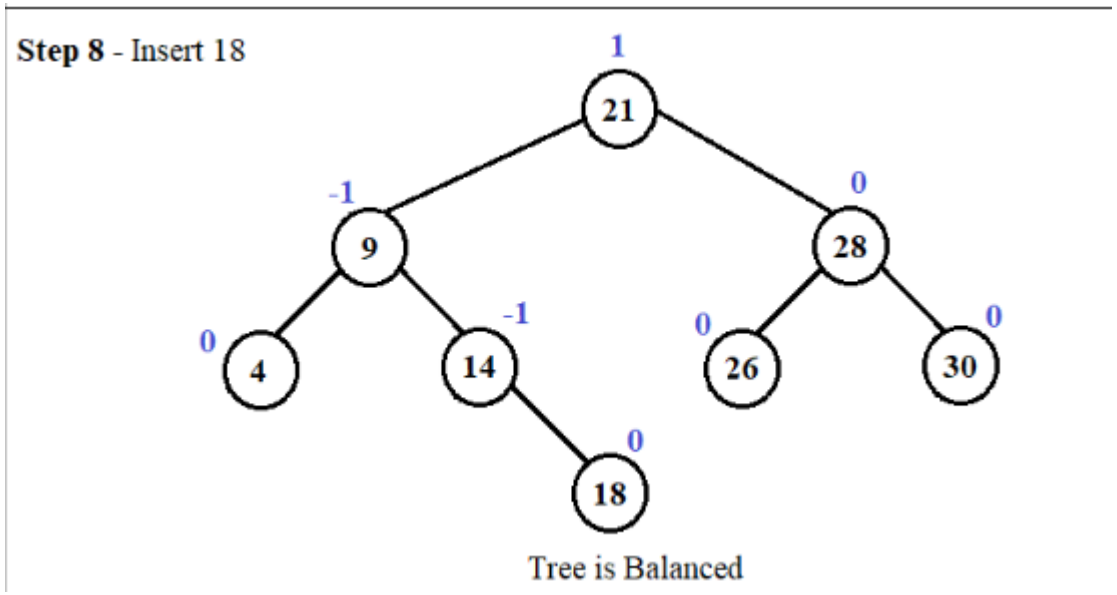




AVL Tree Example

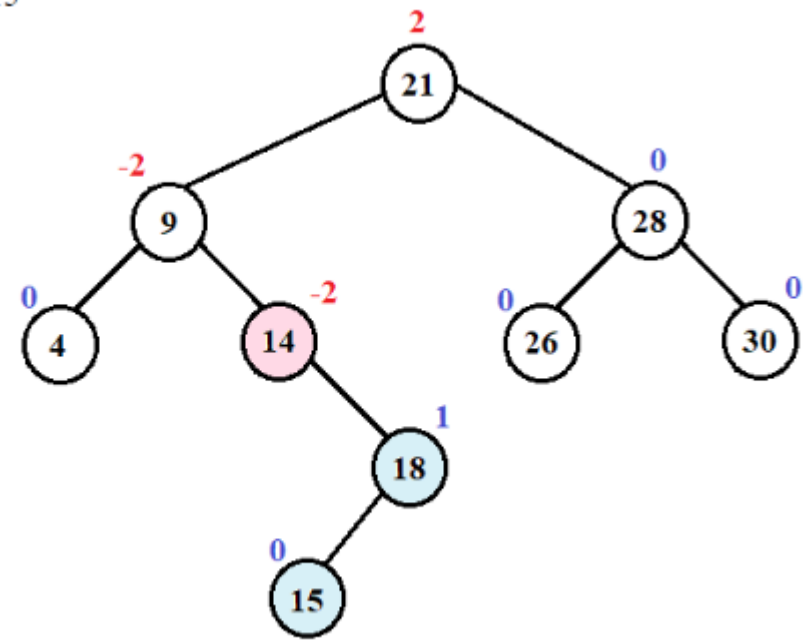


AVL Tree Example



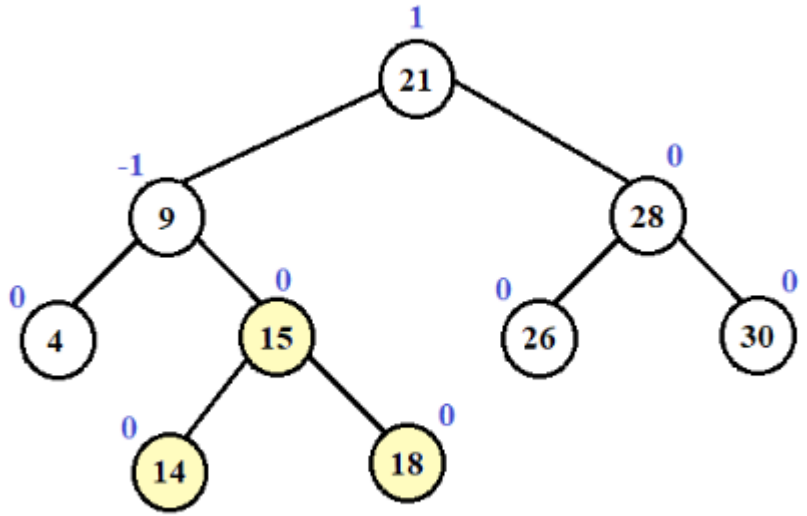
AVL Tree Example

Step 9 - Insert 15



Tree is Not Balanced, Need a Rotation

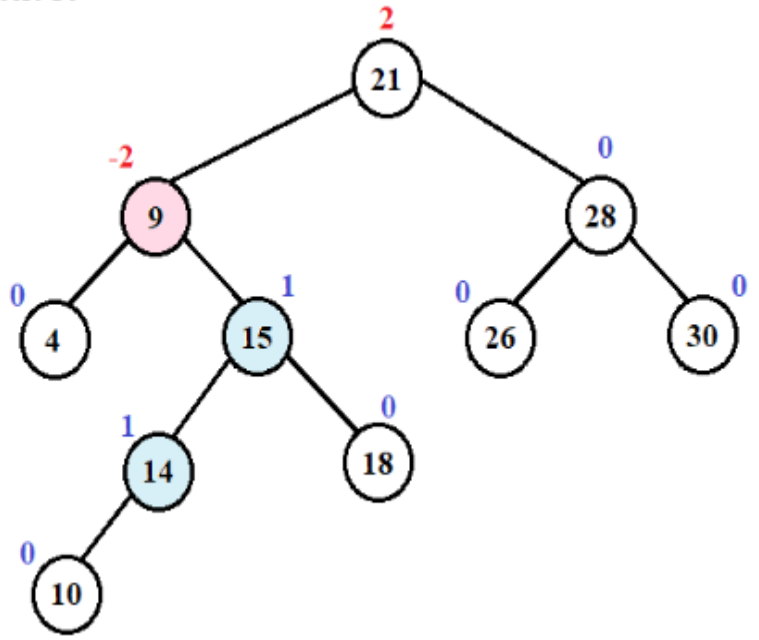
RL Rotation



Tree is Balanced

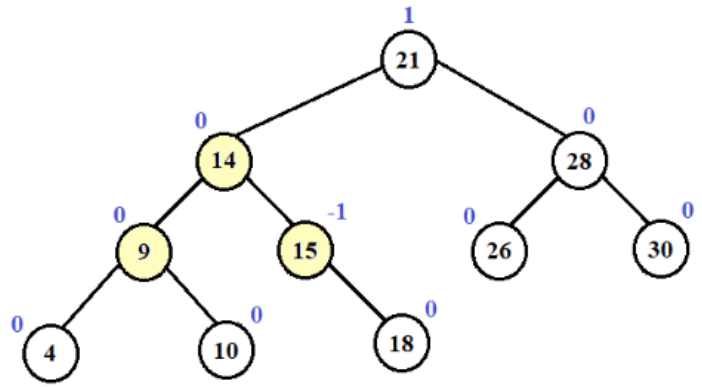
AVL Tree Example

Step 10 - Insert 10



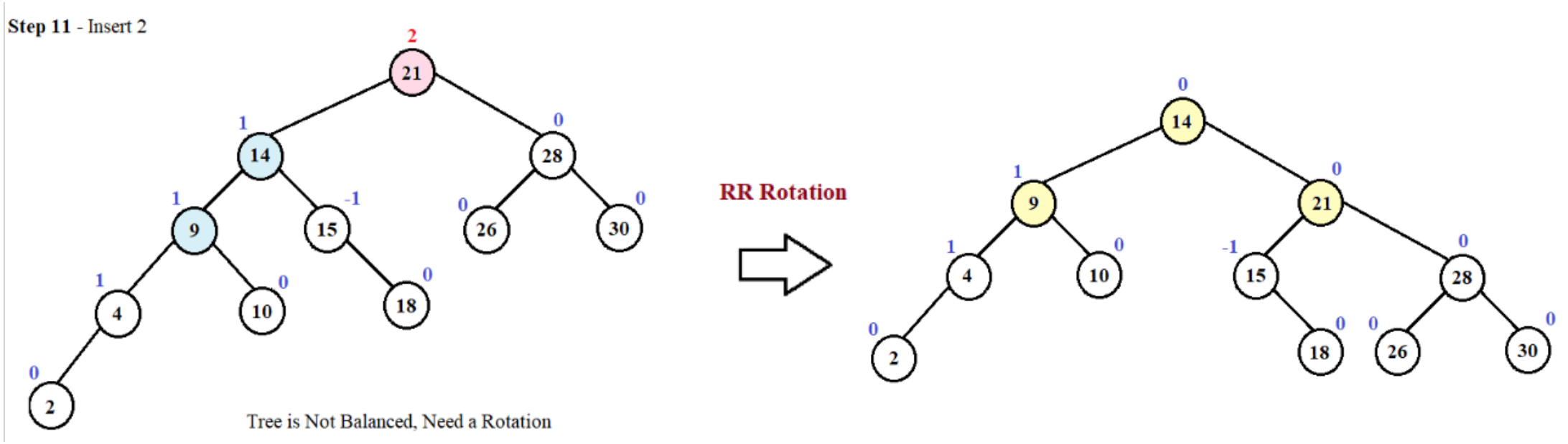
Tree is Not Balanced, Need a Rotation

RL Rotation



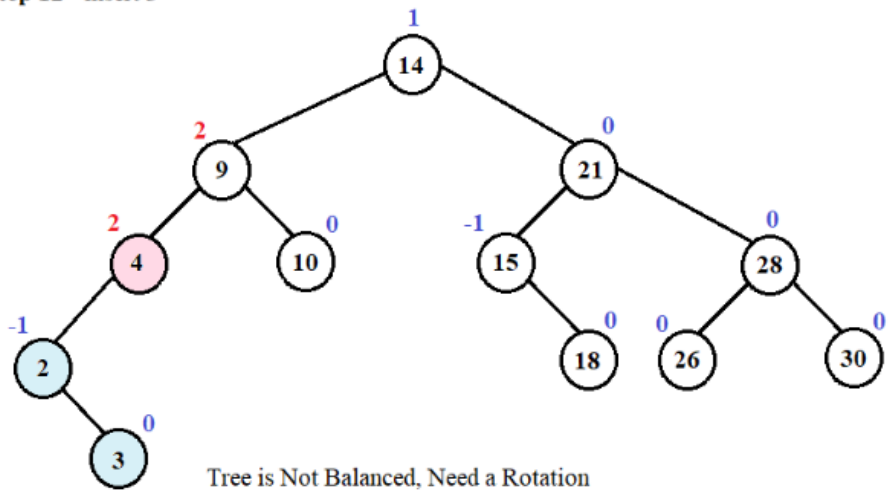
Tree is Balanced

AVL Tree Example



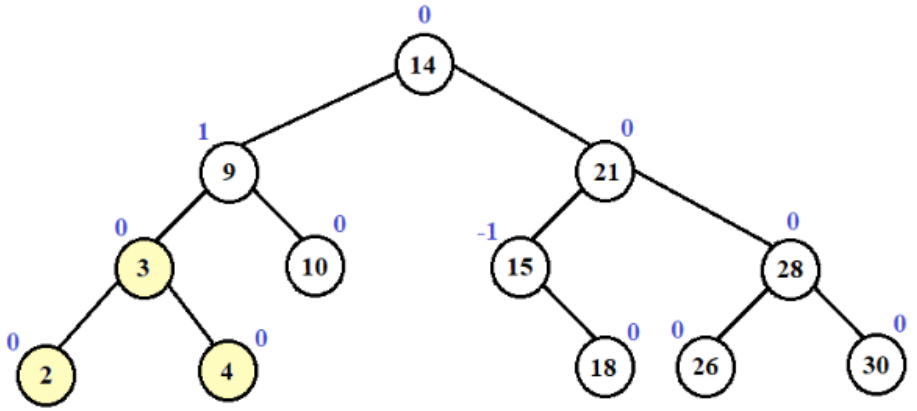
AVL Tree Example

Step 12 - Insert 3



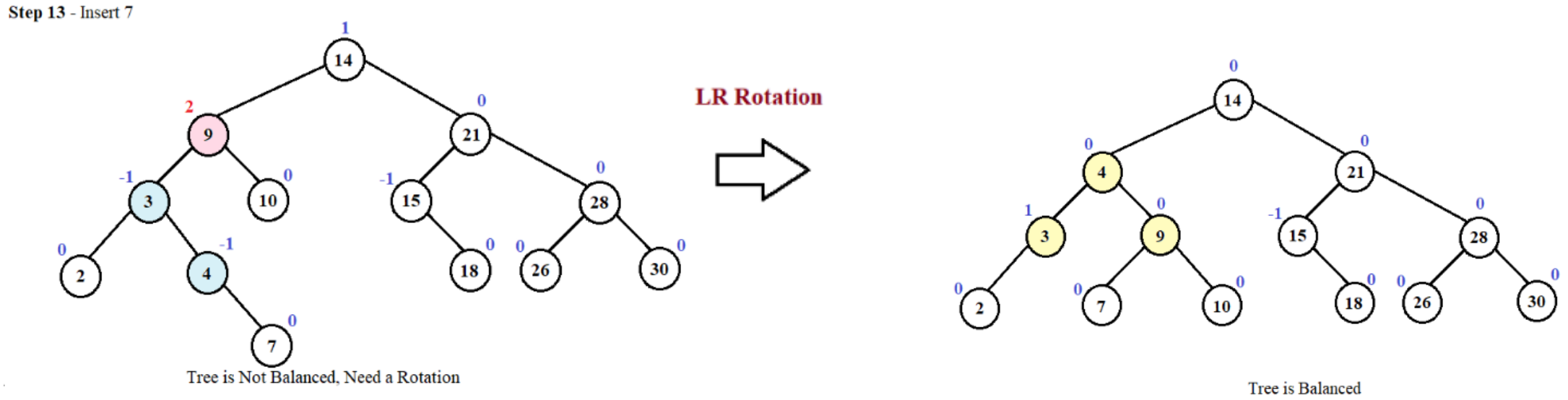
Tree is Not Balanced, Need a Rotation

LR Rotation



Tree is Balanced

AVL Tree Example



Tree is Not Balanced, Need a Rotation

Tree is Balanced

Thank you !!!